

牛客寒假营 6 题解

2026 年 2 月 13 日

写在前面

最后一场比赛。

春节也是要到了，提前祝大家新年快乐！鉴于大家过年可能也会去忙别的事情，这份题解的内容可能很少有人会看完，所以把祝福写在前面。

本场涵盖知识点比较广泛，也是希望目前技能树各异的大家都能从中学到一些东西。

有的同学在这场比赛中的发挥不佳，原因可能是死磕了一些自己先前没有学过的知识点，也有可能是某些题目的 corner case 没有考虑全，总之，暴露问题也是一件好事，大家都是在解决问题的过程中中成长的。

难度顺序大概是：K<G<H<D<A<C<E<B<I<L<F<J

排序难度不严格准确，因为每个人擅长的东西不同，比如可能有的选手擅长写板子，可能觉得 CJ 这类题很简单，有的选手擅长构造，觉得 I 就比较简单。

A 小 L 的三角尺

edu: 考察知识点是贪心，在探索解题方法的时候，把式子列出来可能会帮助你推导到正解的方向。

求 $\sum_{i=1}^n \sqrt{x_i^2 + (y_i - t_i)^2}$ 的最大值，限制条件是 $\sum_{i=1}^n t_i \leq w$ 。

观察 $f(y) = \sqrt{x^2 + y^2}$ 是一个关于 y 的下凸函数，那么对于同一个三角尺来说， y 在减小的过程中，每一步减小带来的收益会逐渐变小，基于这个性质，我们有一个自然的贪心策略：在每一步操作中，总是选择当前所有尺子中，能够带来最大“斜边减少量”的那个尺子进行操作。

本题设置的 w 并不大，是一个可以接受的复杂度参数。

直接堆模拟这个过程即可，复杂度 $O(w \log(n))$ 。

B 小 L 的彩球

edu: 考察知识点组合数学，这个题想教会大家在做题的时候，将模型进行转化可以很大程度上帮助理解。

对于原题意，我们可以转化为这样的理解：

有一个长度为 n 的 01 字符串，问你符合“有 x 个 0， $n-x$ 个 1，01,10 子串共有 t 个”条件的字符串一共有多少个。

这样是不是好思考多了。

那么我们思考，目标子串的个数其实就是 01 段发生转换的次数，那么连续的 0 段和连续的 1 段之和就是 $t + 1$ 段，然后讨论一下段数的奇偶以及 01 段的先后顺序，再使用隔板法求解即可。

C 小 L 的线段树

edu: 线段树是有用的数据结构，本题旨在帮助大家理解线段树。

按照线段树的构建与修改，维护一下每个点是否被毁坏以及需要访问多少未被毁坏的子节点才能获取即可。

D 小 L 的扩展

edu: 考察搜索算法的运用。

在传统的多源起点 bfs 的基础上，手动设置一些优先级别靠后的点，使用优先队列代替队列维护 bfs 即可。

或者直接分成两部分进行扩展，黑色的点扩展到因为蓝色点还没解放而无法扩展时，等待第一个蓝色点解放后扩展，这样的写法也可以。

出这个题的本意是，我发现存在部分初学者，在刚接触的时候过于依赖一些模板，而没有去思考 bfs 的过程，不知道 bfs 为什么用队列从前往后进行维护就是对的。

E 小 L 的空投

edu: 考察知识点为并查集，但是更重要的是想要教会大家逆向思考。

正着做要维护很多信息，比如某些边断了，要怎么删除，要怎么维护连通块，想起来很复杂。

但是我们不妨倒着思考，因为在图论算法中，动态插入一般来说是比动态删除好搞不少的。

我们从所有城市都被淹没开始，沿着时间轴倒着走，那么一个个城市就会逐渐从水里露出来，那么我们只需要用并查集维护一下连通块即可。

F 小 L 的极大团

edu: 考察内容为容斥原理。

纯推式子题，考验大家推式子能力，或者你可以力大砖飞使用多项式做，复杂度应该也是可以通过此题的。

我们需要在 n 个点的图中随机选取 k 条边后，极大团数量的期望。

根据期望的线性性质：

$$E[\text{极大团数量}] = \sum_{S \subseteq V, S \neq \emptyset} P(S \text{ 是一个极大团})$$

由于所有点都是对称的，对于大小为 i 的集合 S ，其成为极大团的概率是相同的。设 $f(i)$ 为某个特定的大小为 i 的集合 S 成为极大团的概率（或满足条件的方案数），则：

$$E = \frac{1}{\binom{M}{k}} \sum_{i=1}^n \binom{n}{i} \times \text{Count}(i)$$

其中 $M = \frac{n(n-1)}{2}$ 是总边数, $\text{Count}(i)$ 是在选取 k 条边的情况下, 使得一个固定的大小为 i 的集合 S 成为极大团的选边方案数。

一个顶点集合 S ($|S| = i$) 是极大团, 需要满足两个条件:

团的条件: S 内部的所有 $\binom{i}{2}$ 条边都必须存在。

极大的条件: 对于 S 外部的任意节点 $v \in V \setminus S$, 不能与 S 中的所有节点都相连 (即 v 与 S 之间的连边数必须小于 i)。

我们要计算满足上述两个条件的方案数 $\text{Count}(i)$ 。首先, 必须选定 S 内部的 $\binom{i}{2}$ 条边。设 $\text{Cost}_{\text{base}} = \binom{i}{2}$ 。如果 $\text{Cost}_{\text{base}} > k$, 则方案数为 0。对于第二个条件 “外部没有节点与 S 全连”, 我们可以使用容斥原理。设 $U = V \setminus S$ 为外部点集, 大小为 $n - i$ 。对于 $v \in U$, 设属性 A_v 表示 “ v 与 S 中的所有 i 个点都相连”。我们需要的是没有任何属性 A_v 成立的方案数。根据容斥原理:

$$\text{Count}(i) = \sum_{j=0}^{n-i} (-1)^j \times \binom{n-i}{j} \times (\text{强制 } j \text{ 个外部点与 } S \text{ 全连的方案数})$$

当我们强制 j 个外部点与 S 全连时, 需要增加的边数为 $j \times i$ 。因此, 总共需要强制选择的边数为:

$$\text{Cost}(i, j) = \binom{i}{2} + j \times i$$

如果在总共 k 条边中, 已经强制选了 $\text{Cost}(i, j)$ 条边, 那么剩下的 $k - \text{Cost}(i, j)$ 条边可以从剩下的 $M - \text{Cost}(i, j)$ 条边中任意选择。方案数为:

$$\binom{M - \text{Cost}(i, j)}{k - \text{Cost}(i, j)}$$

所以:

$$\text{Count}(i) = \sum_{j=0}^{n-i} (-1)^j \binom{n-i}{j} \binom{M - \text{Cost}(i, j)}{k - \text{Cost}(i, j)}$$

直接计算组合数 $\binom{N}{K}$ 其中 $N \approx 10^{10}$ 是不可行的, 但注意到 K (即 $k - \text{Cost}(i, j)$) 非常小 ($\leq k \leq 2 \times 10^5$)。我们可以将组合数变形:

$$\binom{M - C}{k - C} = \binom{(M - k) + (k - C)}{k - C}$$

令 $A = M - k$ (这是一个定值), $r = k - \text{Cost}(i, j)$ (这是当前剩余可自由选择的边数)。我们要计算 $\binom{A+r}{r}$ 。由于 r 最大为 k , 我们可以预处理一个数组 BigComb , 其中 $\text{BigComb}[r] = \binom{A+r}{r}$ 。递推公式: $\text{BigComb}[r] = \text{BigComb}[r-1] \times \frac{A+r}{r}$ 。这样我们可以在 $O(1)$ 时间内获取大组合数的值。

或者你想要力大砖飞:

同样的第一步转化, 你可以在计算 $\text{Count}(i)$ 的时候使用生成函数, 然后使用多项式快速幂求解。

G 小 L 的散步

edu: 双指针的使用。

模拟一下小 L 每步走完之后的脚掌区间，然后随着区间进行维护缝隙的位置，看看这个区间有没有框住缝隙即可。

H 小 L 的数组

edu: 值域 dp。

大家看到数值范围，应该能够敏锐地注意到，这个题的做法是跟值域相关的。

令 $dp[i][j]$ 为操作了 i 次能不能变成 j ，如果 $dp[i-1][j]$ 为 *true*， $dp[i][j \oplus b_i]$ 和 $dp[i][\max(j - a_i, 0)]$ 就为 *true*。

循环到 n 即可。

I 小 L 的构造 2

edu: 构造。

构造 1 详见去年寒假营。

一种可行的构造方式是：

对每 12 个数一组，取出 2 的倍数和 3 的倍数。

我们发现刚好有 8 个。

我们可以这样排列： $1 + 12x, 2 + 12x, 4 + 12x, 7 + 12x, 10 + 12x, 8 + 12x, 11 + 12x, 12 + 12x, 3 + 12x, 5 + 12x, 9 + 12x, 6 + 12x$ 。

边界条件进行特殊讨论即可。

J 小 L 的字符串

edu: 推式子 + 卷积。

我们需要最小化总代价：

$$Cost(k) = k + \sum_{i=0}^{n-1} \text{dist}(s_i, t_{(i+k)\%n})$$

其中单字符距离 $\text{dist}(a, b) = (b - a + 26) \% 26$ 。

将距离公式拆解：若 $t \geq s$ （字符值比较），代价为 $t - s$ 。若 $t < s$ ，代价为 $t - s + 26$ 。这可以统一写作： $(t - s) + 26 \times [s > t]$ ，其中 $[P]$ 当条件 P 成立时为 1，否则为 0。代入总代价公式：

$$Cost(k) = k + \sum (t_{(i+k)\%n} - s_i) + 26 \times \sum [s_i > t_{(i+k)\%n}]$$

$$Cost(k) = \left(k + \sum t - \sum s \right) + 26 \times \text{Count}(k)$$

前面好算，后面需要转化一下后使用 NTT 加速计算对于每个位移 k ，有多少对字符满足 $s_i > t_{(i+k)\%n}$ 即可，学习理解完 NTT 模板后应该就会了。

K 小 L 的游戏 1

edu: 取模

注意到得到的结果只有可能是 $(m+n)*k$ 和 $m+(m+n)*k$ ，用 z 对 $m+n$ 取模看结果是多少即可。

L 小 L 的游戏 2

edu: 矩阵快速幂

K 的加强版，注意到 $t = \max(m_1, m_2, n_1, n_2)$ 很小。

设 $dp[i][0]$ 为小 L 最后一次操作后等于 i 的概率， $dp[i][1]$ 为 fallleaves01 最后一次操作后等于 i 的概率，状态转移方程为：

$$dp[i][0] = dp[i - m_1][1] * p + dp[i - m_2][1] * (1 - p)$$

$$dp[i][1] = dp[i - n_1][0] * q + dp[i - n_2][0] * (1 - q)$$

鉴于 t 很小，于是我们只关心 $z - t$ 到 $z - 1$ 的时候小 L 能不能一步走到 z 以及 z 以上。

于是我们可以使用矩阵加速这个 dp 过程，状态转移矩阵 M 的设置 $2t * 2t$ ：

$$i = j + 2 \text{ 时, } M[i][j] = 1;$$

$$M[0][2m_1 - 1] = p, \quad M[0][2m_2 - 1] = 1 - p,$$

$$M[1][2n_1 - 1] = q, \quad M[1][2n_2 - 1] = 1 - q,$$

其它位置置 0。

向量设置为：

$$B = \begin{bmatrix} dp[t][0] \\ dp[t][1] \\ dp[t-1][0] \\ dp[t-1][1] \\ \vdots \\ dp[1][0] \\ dp[1][1] \end{bmatrix}$$

矩阵快速幂后枚举数组能不能一步到 z 即可。

写在最后

新年快乐！